

Regression Training & Evaluation Lab (Practical)

Welcome to the practical lab for Regression Training & Evaluation! In this session, you will write a standalone Python script to train a **Linear Regression** model using **Scikit-Learn**, extract and analyze its coefficients and intercept using matrix mathematics on sample data, and evaluate its performance using MAE, MSE, and R^2 Score.

1. Concept Overview

In this lab, you will:

- Split your dataset into training and testing subsets using `train_test_split`.
- Fit a `LinearRegression` model to learn the relationship between input features and target values.
- Mathematically interpret the learned **intercept** (β_0) and **coefficients** (β_1, β_2).
- Evaluate model accuracy using **Mean Absolute Error (MAE)**, **Mean Squared Error (MSE)**, and R^2 **Score**.

2. Step-by-Step Lab Instructions

Step 1: Create the Training Script (`train_regression.py`)

Create a new file named `train_regression.py` in your project folder and add the following code:

```
# train_regression.py
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# 1. Create a sample dataset (e.g., House Size & Rooms vs Price)
data = {
```

```

    "feature1": [1200, 1500, 1800, 2000, 2200, 2500, 2800, 3000], # Square
Footage
    "feature2": [2, 3, 3, 4, 4, 5, 5, 5], # Number
of Rooms
    "target_price": [180000, 220000, 260000, 290000, 310000, 350000,
390000, 420000] # Price ($)
}

df = pd.DataFrame(data)

# Separate features (X) and target (y)
X = df[["feature1", "feature2"]]
y = df["target_price"]

# 2. Split dataset into Training (80%) and Testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=42)

# 3. Initialize and train the Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)

# 4. Extract Intercept and Coefficients
print("=== Model Parameters ===")
print(f"Intercept (Beta 0): {model.intercept_:.2f}")
print(f"Coefficients (Beta 1, Beta 2): {model.coef_}\n")

# 5. Make predictions on the test set
y_pred = model.predict(X_test)

# 6. Evaluate Performance Metrics
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("=== Evaluation Metrics ===")
print(f"Mean Absolute Error (MAE): ${mae:,.2f}")
print(f"Mean Squared Error (MSE): {mse:,.2f}")
print(f"R2 Score: {r2:.4f}")

```

Step 2: Run Your Script

Open your terminal, ensure your virtual environment is active, and execute the script:

```
python train_regression.py
```

3. Mathematical & Matrix Walkthrough on Sample Data

When Scikit-Learn runs `model.fit(X_train, y_train)`, it solves the Ordinary Least Squares (OLS) optimization problem using matrix algebra:

$$\beta = (X^T X)^{-1} X^T Y$$

A. Matrix Structure of Our Sample Data

Let's look at how the design matrix X and target vector Y are structured under the hood:

- **Feature Matrix (X):** Contains columns for `feature1` and `feature2` (plus an implicit column of ones for the intercept).
- **Target Vector (Y):** Contains the `target_price` values.

B. Interpreting the Output

When you run the script, you will see output similar to this:

- **Intercept (β_0):** Represents the base price of a property when all features equal zero.
- **Coefficients (β_1, β_2):**
 - The coefficient for `feature1` tells you the exact dollar increase expected per additional square foot.
 - The coefficient for `feature2` tells you the price increase per additional room.

C. Reading the Evaluation Metrics

- **MAE:** Tells you, on average, how many dollars off your model's price predictions are.
 - **R^2 Score:** A value close to 1.0 (e.g., 0.95+) indicates that your model explains almost all the variance in house prices based on square footage and rooms!
-

4. Reference Links & Further Reading

- **Scikit-Learn LinearRegression API Reference:** Detailed attributes and methods for linear models.
 - **Scikit-Learn Model Evaluation Metrics:** Guide on calculating MAE, MSE, and R^2 scores in Python.
-

5. Mathematical Walkthrough

```
import numpy as np
```

```
feature1 = [1200, 1500, 1800, 2000, 2200, 2500, 2800, 3000]  
feature2 = [2, 3, 3, 4, 4, 5, 5, 5]  
target_price = [180000, 220000, 260000, 290000, 310000, 350000, 390000, 420000]
```

```

# Construct X (adding intercept column of 1s)
X = np.column_stack((np.ones(len(feature1)), feature1, feature2))
Y = np.array(target_price).reshape(-1, 1)

# Computations
X_T = X.T
X_T_X = X_T.dot(X)
X_T_X_inv = np.linalg.inv(X_T_X)
X_T_Y = X_T.dot(Y)
beta = X_T_X_inv.dot(X_T_Y)

print("X:\n", X)
print("Y:\n", Y)
print("X_T_X:\n", X_T_X)
print("X_T_X_inv:\n", np.array2string(X_T_X_inv, formatter=
{'float_kind': lambda x: "%.5e" % x}))
print("X_T_Y:\n", X_T_Y)
print("Beta:\n", beta)

X:
[[1.0e+00 1.2e+03 2.0e+00]
 [1.0e+00 1.5e+03 3.0e+00]
 [1.0e+00 1.8e+03 3.0e+00]
 [1.0e+00 2.0e+03 4.0e+00]
 [1.0e+00 2.2e+03 4.0e+00]
 [1.0e+00 2.5e+03 5.0e+00]
 [1.0e+00 2.8e+03 5.0e+00]
 [1.0e+00 3.0e+03 5.0e+00]]
Y:
[[180000]
 [220000]
 [260000]
 [290000]
 [310000]
 [350000]
 [390000]
 [420000]]
X_T_X:
[[8.000e+00 1.700e+04 3.100e+01]
 [1.700e+04 3.886e+07 7.060e+04]
 [3.100e+01 7.060e+04 1.290e+02]]
X_T_X_inv:
[[1.83440e+00 -2.82413e-04 -2.86264e-01]
 [-2.82413e-04 4.55712e-06 -2.42619e-03]
 [-2.86264e-01 -2.42619e-03 1.40436e+00]]
X_T_Y:
[[2.420e+06]
 [5.503e+09]
 [1.000e+07]]
Beta:
[[22490.37227214]]

```

[132.54172015]
[-423.62002567]]

The **Normal Equation** — $\beta = (X^T X)^{-1} X^T Y$ — is the closed-form matrix solution for multiple linear regression. Instead of using an iterative algorithm like gradient descent to find the line of best fit, this formula uses linear algebra to jump straight to the exact optimal weights by minimizing the sum of squared errors.

Here is how the components map to your regression model:

- **X (Design Matrix):** Your features (Square Footage and Rooms), prepended with a column of 1s to calculate the baseline starting price (β_0 or y-intercept).
- **Y (Target Vector):** The actual house prices you want to predict.
- $X^T X$: Captures the variance within your features and the covariance between them.
- $(X^T X)^{-1}$: Inverting this matrix essentially "divides" by the feature variance to isolate the pure feature weights.
- $X^T Y$: Captures the covariance between your features and the target prices.

Step 1: Build X and Y

To start, we construct the 8×3 matrix X and the 8×1 vector Y using your raw data.

$$X = \begin{bmatrix} 1 & 1200 & 2 \\ 1 & 1500 & 3 \\ 1 & 1800 & 3 \\ 1 & 2000 & 4 \\ 1 & 2200 & 4 \\ 1 & 2500 & 5 \\ 1 & 2800 & 5 \\ 1 & 3000 & 5 \end{bmatrix}, \quad Y = \begin{bmatrix} 180000 \\ 220000 \\ 260000 \\ 290000 \\ 310000 \\ 350000 \\ 390000 \\ 420000 \end{bmatrix}$$

Step 2: Compute $X^T X$ and $X^T Y$

Next, we multiply the transposed matrix X^T by the original X . This compresses our 8 data points into a compact 3×3 matrix representing feature relationships.

$$X^T X = \begin{bmatrix} 8 & 17000 & 31 \\ 17000 & 38860000 & 70600 \\ 31 & 70600 & 129 \end{bmatrix}$$

We do the same for $X^T Y$ to measure how the features relate to the target price:

$$X^T Y = \begin{bmatrix} 2420000 \\ 550300000 \\ 10000000 \end{bmatrix}$$

Step 3: Invert and Multiply for β

Finally, we calculate the inverse of the square matrix $(X^T X)^{-1}$ and multiply it by $X^T Y$. This produces the exact coefficient vector β , which contains our optimal weights $[\beta_0, \beta_1, \beta_2]$.

Using exact matrix algebra on your data, the result is:

- β_0 (**Intercept**): 22490.37
- β_1 (**Square Footage**): 132.54
- β_2 (**Rooms**): -423.62

The Final Equation:

$$\text{Price} = 22490.37 + 132.54(\text{SqFt}) - 423.62(\text{Rooms})$$

Key insight: The coefficient for rooms is slightly negative. Because square footage and room count are highly correlated in your data, the math determines that if two houses have the *exact same square footage*, slicing that same floor plan into *more* rooms means the rooms are smaller, which slightly decreases the model's price prediction.
